

Session 4: Functions, Lambdas & Functional Programming

Take-Home Practice Problems

Introduction

Welcome to Session 04 Practice!

These problems will help you master the key concepts from Session 04:

Topics Covered:

- Functions with parameters and return values
- Lambda functions for quick operations
- Built-in functions: `map()`, `filter()`, `sum()`
- Working with lists: `append()`, `extend()`
- Data combination: `zip()`, `enumerate()`
- Custom sorting with `sorted()` and `key` parameter

What Makes These Problems Better:

- ✓ **Clear requirements** - Each function has specific inputs and outputs
- ✓ **Built-in tests** - Run your code and see if it passes!
- ✓ **Formatted output** - All printing is handled for you
- ✓ **Focus on logic** - Just write the function bodies

How to Complete the Problems

1. Open the `take_home_questions` folder in the `session04` directory
2. You'll find 3 Python files, one for each question
3. Each file has:
 - Function templates with clear documentation
 - TODO comments showing where to write your code
 - Test code that runs automatically
 - Expected output examples
4. Complete each function by replacing `pass` with your code

5. Run the file to test your implementation
6. Don't modify the test code at the bottom!

Question 1: Data Processor

File: `question1_data_processor.py`

Topics: `map()`, `filter()`, lambda functions

What You'll Do:

Implement three functions to process lists of numbers:

1. `square_all()` - Use `map()` and lambda to square all numbers
2. `filter_even()` - Use `filter()` and lambda to keep only even numbers
3. `sum_above_threshold()` - Combine `filter()`, lambda, and `sum()`

Key Concepts:

- Using `map()` to transform all items
- Using `filter()` to select items matching a condition
- Writing lambda functions: `lambda x: x ** 2`
- Converting map/filter results to lists

Example Output:

```
=====
SESSION 04 - QUESTION 1: DATA PROCESSOR
=====

Original numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

TEST 1: Square all numbers
-----
Result: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Expected: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
Status: PASS
```

Question 2: Student Ranker

File: `question2_student_ranker.py`

Topics: `zip()`, `enumerate()`, `sorted()` with key

What You'll Do:

Implement three functions to manage student data:

1. `create_student_records()` - Use `zip()` to combine names, scores, ages
2. `rank_by_score()` - Use `sorted()` with lambda key to rank by score
3. `print_numbered_list()` - Use `enumerate()` for numbered output

Key Concepts:

- Using `zip()` to combine multiple lists
- Custom sorting with `sorted(key=lambda x: x[1])`
- Using `enumerate(start=1)` for numbered lists
- Working with tuples: `(name, score, age)`

Example Output:

```
=====
SESSION 04 - QUESTION 2: STUDENT RANKER
=====

Input Data:
Names:  ['Alice', 'Bob', 'Charlie', 'Diana', 'Eve']
Scores: [85, 92, 78, 95, 88]
Ages:   [20, 21, 19, 22, 20]

TEST 1: Create student records using zip()
-----
Result: [('Alice', 85, 20), ('Bob', 92, 21), ...]
Status: PASS
```

Question 3: Shopping List Manager

File: `question3_shopping_list.py`

Topics: Lists, `append()`, `extend()`, `sum()`, `min()`, `max()`

What You'll Do:

Implement four functions to manage a shopping list:

1. `add_single_item()` - Use `append()` to add one item
2. `add_multiple_items()` - Use `extend()` to add many items
3. `calculate_total()` - Use `sum()` to add up prices
4. `find_price_range()` - Use `min()` and `max()` to find price range

Key Concepts:

- **Critical:** `append()` for one item vs `extend()` for multiple
- Using built-in functions: `sum()`, `min()`, `max()`
- Understanding that these functions modify lists **in place**
- Returning tuples: `return (min_val, max_val)`

Example Output:

```
=====
SESSION 04 - QUESTION 3: SHOPPING LIST MANAGER
=====

TEST 1: Add single item using append()
-----
Initial list: ['milk', 'eggs']
After adding 'bread': ['milk', 'eggs', 'bread']
Status: PASS
```

Success Criteria

✓ You've Mastered Session 04 If:

- All tests pass with `PASS` status
- You understand when to use `map()` vs `filter()`
- You can write lambda functions confidently
- You know the difference between `append()` and `extend()`
- You can use `zip()` to combine lists
- You understand `sorted()` with custom key parameter
- You can use `enumerate()` for numbered lists

Common Mistakes to Avoid

Watch Out!

1. Forgetting list() around map/filter

```
1 # Wrong
2 result = map(lambda x: x**2, numbers) # Returns map object
3
4 # Right
5 result = list(map(lambda x: x**2, numbers)) # Returns list
6
```

2. Using append() instead of extend() for multiple items

```
1 # Wrong - Creates nested list!
2 items = [1, 2]
3 items.append([3, 4]) # [1, 2, [3, 4]]
4
5 # Right
6 items = [1, 2]
7 items.extend([3, 4]) # [1, 2, 3, 4]
8
```

3. Incorrect lambda syntax for sorted()

```
1 # Wrong
2 sorted(students, key=lambda x: x[1]) # If sorting tuples
3
4 # Right (access second element)
5 sorted(students, key=lambda x: x[1], reverse=True)
6
```

4. Not using enumerate() start parameter

```
1 # If you want numbering from 1, not 0:
2 for i, item in enumerate(items, start=1):
3     print(f"{i}. {item}")
4
```

Key Concepts Reference

Quick Reference

map() - Transform all items:

```
1 squared = list(map(lambda x: x**2, [1, 2, 3])) # [1, 4, 9]
```

filter() - Keep matching items:

```
1 evens = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4])) # [2, 4]
```

zip() - Combine lists:

```
1 pairs = list(zip(['a', 'b'], [1, 2])) # [('a', 1), ('b', 2)]
```

enumerate() - Get index + value:

```
1 for i, val in enumerate(['a', 'b'], start=1):  
2     print(f"{i}. {val}") # 1. a, 2. b
```

sorted() with key:

```
1 students = [('Bob', 85), ('Alice', 92)]  
2 sorted_students = sorted(students, key=lambda x: x[1], reverse=True)  
3 # [('Alice', 92), ('Bob', 85)]
```

append() vs **extend()**:

```
1 # append() - adds ONE item (even if it's a list)  
2 lst = [1, 2]  
3 lst.append(3) # [1, 2, 3]  
4 lst.append([4, 5]) # [1, 2, 3, [4, 5]]  
5  
6 # extend() - adds EACH item from an iterable  
7 lst = [1, 2]  
8 lst.extend([3, 4]) # [1, 2, 3, 4]
```

Testing Your Code

To run each question:

```
cd session04/take_home_questions  
python question1_data_processor.py  
python question2_student_ranker.py  
python question3_shopping_list.py
```

Each file will:

- Run multiple tests automatically
- Show expected vs actual output
- Display PASS or FAIL for each test
- Include bonus examples to explore

Next Steps After Completion

1. **Experiment:** Try modifying the test data to see how your functions handle different inputs
2. **Challenge yourself:** Look at the bonus sections in each file
3. **Review:** If any tests fail, review the function documentation and hints
4. **Practice:** Try writing your own test cases
5. **Prepare:** These concepts are essential for Session 05!

Final Encouragement

Remember:

- **Read carefully:** Each function has clear documentation and examples
- **Start simple:** Get the basic logic working first
- **Test often:** Run your code after each function
- **Use hints:** Check the docstrings for guidance
- **Ask questions:** If you're stuck, reach out!

These problems are designed to be clear, focused, and achievable. The boilerplate code handles all the formatting - you just need to implement the core logic. You've got this!



Questions? Need help? Don't hesitate to reach out!